



ISPTEC

Instituto Superior Politécnico de Tecnologias e Ciências

SINGLE PAGE APPLICATIONS

Autor: Sediangani Sofrimento

Área: DET

□ A World Wide Web

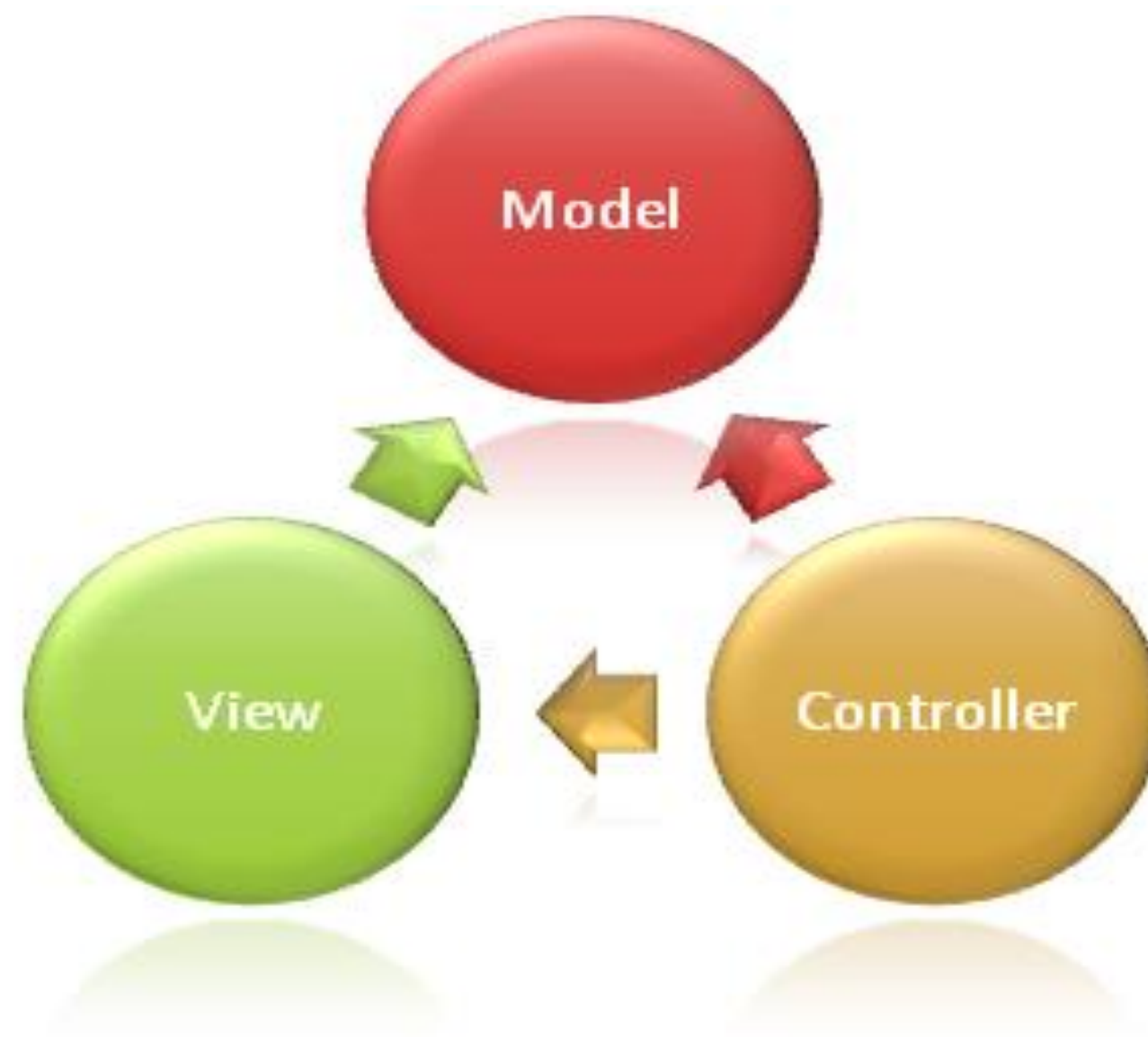
- Sistema de partilha de informação baseado na Internet(a rede) com 3 componentes importantes:
 - um protocolo de transmissão de dados – **HTTP**
 - um sistema de endereçamento próprio – **URL**
 - uma linguagem de marcação, para visualização dos documentos – **HTML**



❖ MVC – MODEL VIEW CONTROLLER

- Com o aumento da complexidade das aplicações desenvolvidas, torna-se fundamental a separação entre os dados (Model) e o layout (View).
 - Desta forma, alterações feitas no layout não afectam a manipulação de dados, e estes poderão ser reorganizados sem alterar o layout.
- **MVC** é o acrónimo para Model-View-Controller, que é uma arquitectura que visa fornecer uma maneira de separar as funcionalidades envolvidas em uma aplicação de maneira a melhorar a manutenção do sistema.
- Actualmente o **MVC** é considerado um padrão em engenharia de software, utilizado praticamente por todas as frameworks modernas e nos principais projectos em todo o mundo.





➤ **M** – Model

- Na arquitectura MVC o **model** representa os dados da aplicação e as regras do negócio que definem o acesso e a modificação dos dados.

➤ **V** – View

- Renderiza(apresenta) o conteúdo de uma parte particular do **Model** e encaminha para o **Controller** as acções do utilizador.

➤ **C** – Controller

- Define o comportamento da aplicação, interpreta as acções(cliques de botões ou selecções de menus) do utilizador e as mapeia para chamadas do **Model**



➤ **M** – Model (camada de lógica da aplicação)

- ❑ Responsável pelas regras de negócios (validações, cálculos etc)
- ❑ Persistência com a base de dados e as classes de entidades
- ❑ Recebe as requisições vindas do **Controller** e gera respostas a partir destas requisições

➤ **V** – View (camada de apresentação)

- ❑ Inclui os elementos de exibição no cliente
 - HTML, XML, JSP, Javascript etc.
- ❑ É utilizada para receber as entradas de dados e apresentar o resultado.

➤ **C** – Controller (camada de controlo)

- ❑ Interpreta as acções do utilizador e mapeia-as para chamadas do **Model**
- ❑ Responsável pelo fluxo da aplicação
- ❑ Intermediário entre a camada de apresentação (**View**) e a camada de negócios (**Model**)



❖ Vantagens

- Separação muito clara entre as camadas de visualização e regras de negócios;
- Fácil manutenção do sistema;
- Grande aproveitamento de código (camada de modelo);
- Permite o desenvolvimento em paralelo para o **modelo**, **vista** e **controlador** pois são independentes;
- As alterações na **camada da vista** não afectam as regras de negócios já implementadas **na camada de modelo**;
- Permite o desenvolvimento, testes e manutenção de forma isolada entre as camadas;
- Código limpo;
- Melhor e mais fácil entendimento do projecto para novos programadores que não participaram na sua criação;
- Torna a aplicação escalável.



❖ Desvantagens

- Em sistemas de baixa complexidade, o MVC pode criar uma complexidade desnecessária;
- Exige muita disciplina dos desenvolvedores em relação à separação das camadas;
- Requer maior tempo para modelação do sistema.

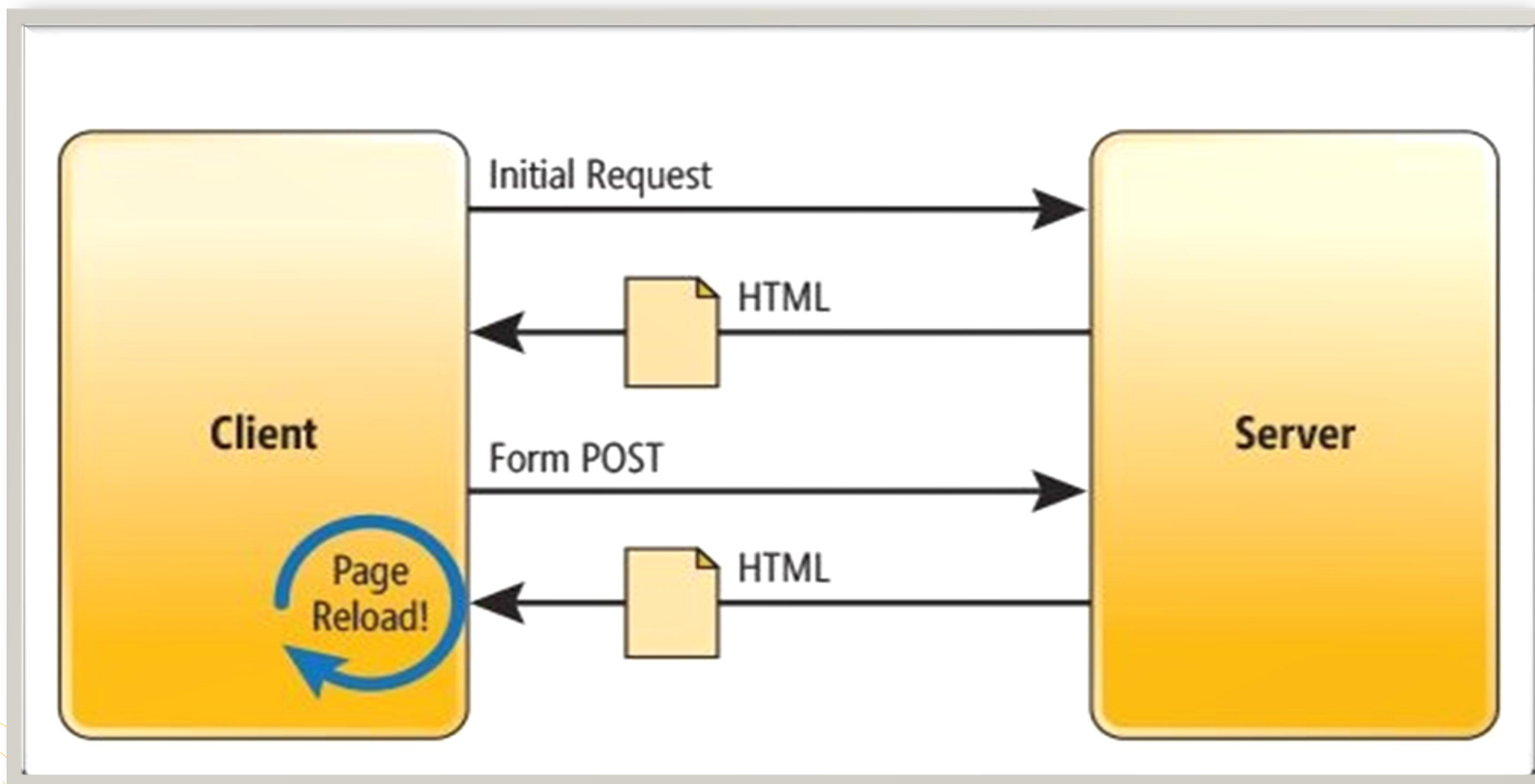


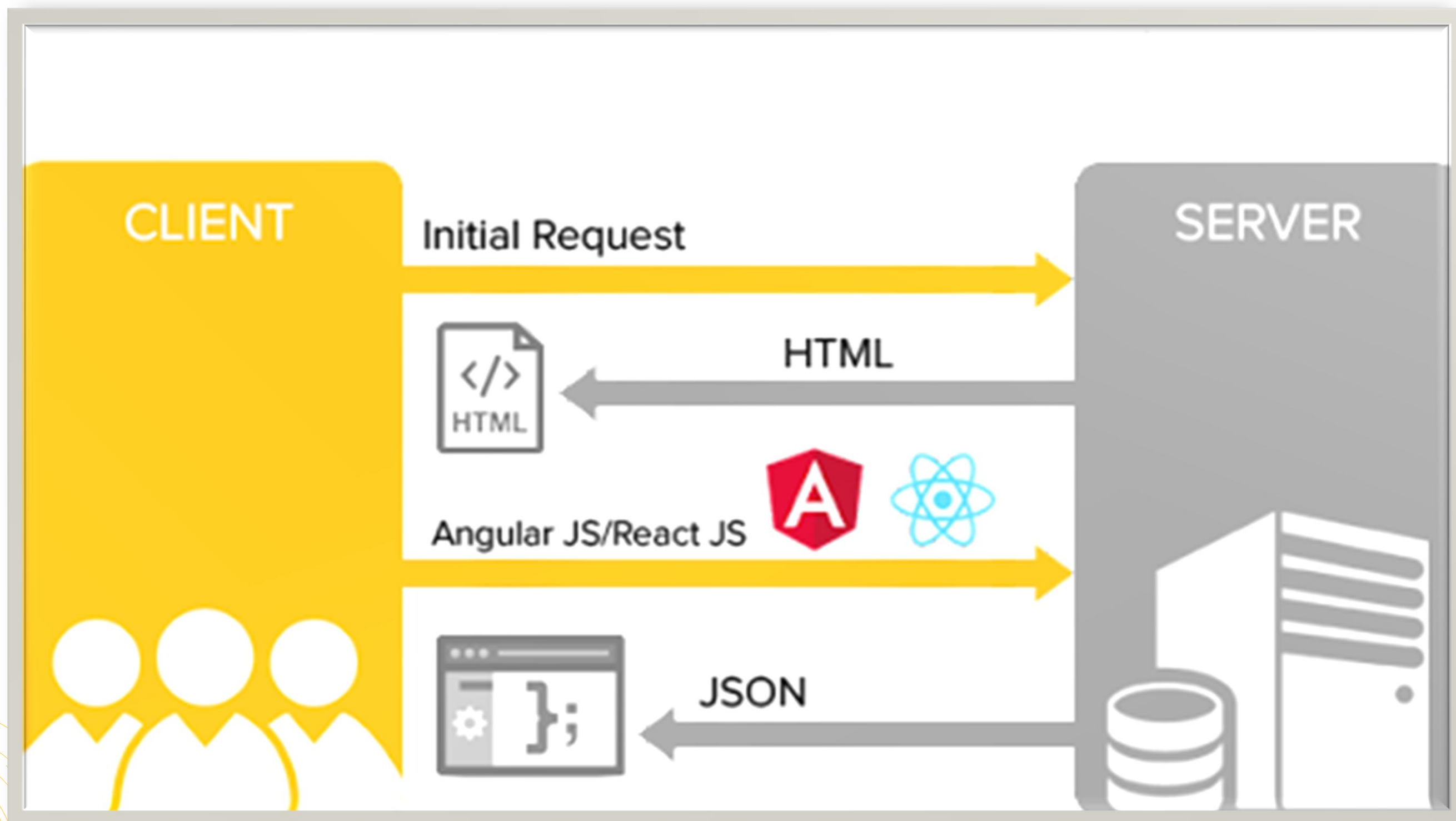
❖ Alguns Frameworks



- É uma abordagem moderna de construção de aplicações Web
- A aplicação interage com o servidor, por requisições **AJAX**
- Os dados são transmitidos utilizando **JSON** ou **XML**







CLIENTE – SIDE

- Todos os processos são executados no lado do cliente.
- A comunicação com o **back-end** é feita através de chamadas **AJAX** para end-points de uma **API**
- Funciona independente do back-end, seja ele desenvolvido em .Net, java, node, php ou ruby
- Facilita a migração de tecnologias.

LOAD(Carregamento)

- Muitas vezes é feito apenas uma vez
- Carregamento de assets como
 - css,
 - javascript,
 - imagens,
 - bootstrap



ROUTER(Rotas)

- Importantíssimas nas SPA's
- Responsáveis pela navegação entre ecrãs
- Mesmo após um refresh permite que a página volte(quase) ao seu estado anterior

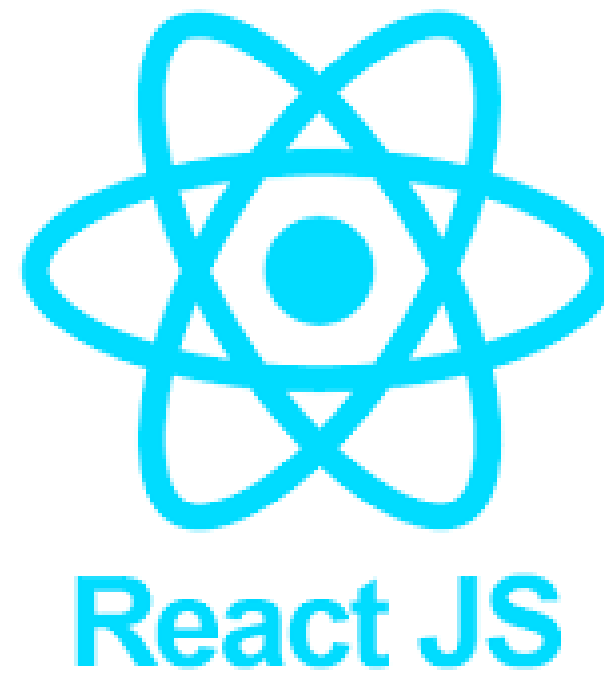
ARMAZENAMENTO

- Nas SPA's é comum guardar os dados localmente no browser do utilizador.
- Geralmente é armazenado um **Token** e outras informações de identificação do utilizador.
- É incomum utilizar-se **Cookies** ou **Sessions** para tal o mais comum é utilizar-se o **localStorage**.



- ❑ Impede o Search Engine Optimization (SEO)
 - as SPA's utilizam frequentemente apenas um URL o que causa problemas de SEO porque cada conteúdo requer um URL único
- ❑ Sobrecarga do Cliente
- ❑ Problemas de Segurança.





□ Eficiência

- Projectos que costumavam levar meses e centenas de linhas de código agora podem ser realizados muito mais rápido com padrões e funções bem estruturadas.

□ Segurança

- As principais estruturas de javascript tem acordos de segurança firmes e são suportadas por grandes comunidades

□ Custo

- A maior parte das frameworks são Open Source





A photograph of a modern campus entrance. In the foreground, a yellow rectangular sign with the text 'ISPTEC' in white capital letters sits on a grassy area. Behind it, a tall, abstract yellow sculpture with two interlocking loops stands on a silver pole. To the right, a modern, white, multi-story building with large glass windows and a flat roof is visible. A paved walkway leads towards the building. In the background, there are trees, a fence, and a clear blue sky. A yellow geometric graphic is in the top right corner.

ATÉ A PRÓXIMA AULA!

Volenti Nihil Difficili - “A quem quer, nada é difícil”